ASSESSMENT - 3

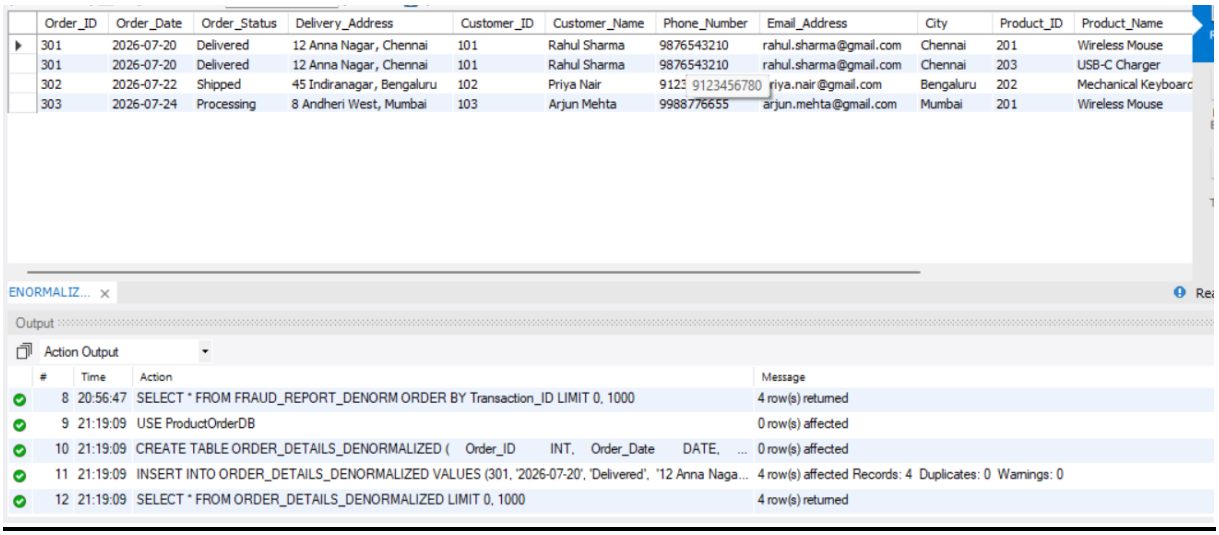
DATABASE SYSTEMS - ABHAY KUMAR 25BCE2537

PRODUCT ORDER MANAGEMENT SYSTEM

DENORMALIZED TABLE:-

```sql
-- PRODUCT ORDER MANAGEMENT SYSTEM
-- Name: ABHAY KUMAR    |   Reg No: 25BCE2537
-- DENORMALIZED TABLE (flat, single-table )
-- Combines Customers + Products + Orders + Order_Items into
-- ONE table, so it contains repeating groups and redundant data
USE ProductOrderDB;
-- STEP 1: CREATE THE DENORMALIZED TABLE-
CREATE TABLE ORDER_DETAILS_DENORMALIZED (
    Order_ID            INT,
    Order_Date          DATE,
    Order_Status        VARCHAR(20),
    Delivery_Address    VARCHAR(100),
    Customer_ID         INT,
    Customer_Name       VARCHAR(50),
    Phone_Number        VARCHAR(15),
    Email_Address       VARCHAR(60),
    City                VARCHAR(30),
    Product_ID          INT,
    Product_Name        VARCHAR(50),
    Category            VARCHAR(30),
    Unit_Price          DECIMAL(10,2),
    Quantity            INT,
    Price_Each          DECIMAL(10,2)
);
-- ------------------------------------------------------------
-- STEP 2: INSERT VALUES (one row per order item, exactly like
-- joining CUSTOMERS + ORDERS + ORDER_ITEMS + PRODUCTS)
-- ------------------------------------------------------------
INSERT INTO ORDER_DETAILS_DENORMALIZED VALUES
(301, '2026-07-20', 'Delivered',  '12 Anna Nagar, Chennai',     101, 'Rahul Sharma', '9876543210', 'rahul.sharma@gmail.com', 'Chennai',
     201, 'Wireless Mouse',       'Electronics', 699.00,  2, 699.00),
(301, '2026-07-20', 'Delivered',  '12 Anna Nagar, Chennai',     101, 'Rahul Sharma', '9876543210', 'rahul.sharma@gmail.com', 'Chennai',
     203, 'USB-C Charger',        'Accessories', 999.00,  1, 999.00),
(302, '2026-07-22', 'Shipped',    '45 Indiranagar, Bengaluru',  102, 'Priya Nair',   '9123456780', 'priya.nair@gmail.com',   'Bengaluru',
     202, 'Mechanical Keyboard',  'Electronics', 2499.00, 1, 2499.00),
(303, '2026-07-24', 'Processing', '8 Andheri West, Mumbai',     103, 'Arjun Mehta',  '9988776655', 'arjun.mehta@gmail.com',  'Mumbai',
     201, 'Wireless Mouse',       'Electronics', 699.00,  1, 699.00);
-- ------------------------------------------------------------
-- STEP 3: VIEW THE DENORMALIZED DATA
-- ------------------------------------------------------------
SELECT * FROM ORDER_DETAILS_DENORMALIZED;
-- 1. Customer_Name, Phone_Number, Email_Address, City repeat for
--    every order the same customer places (rows 1 & 2 both show
--    Rahul Sharma's full details) -> redundancy / update anomaly,
--    the root problem 1NF/2NF/3NF fix.
-- 2. Product_Name, Category, Unit_Price repeat every time the same
--    Product_ID is ordered (Wireless Mouse appears in row 1 and 4)
--    -> transitive dependency, fixed by moving Products to its own
--    table (which Assessment 2 already did correctly).
-- 3. This single table mixes FOUR different entities (Customer,
--    Order, Product, Order Item) in one row - a classic
--    denormalized / un-normalized design used as the "before"
--    state for the 1NF, 2NF and 3NF corrections shown next.
-- ------------------------------------------------------------
```

DENORMALIZATION RESULT:-



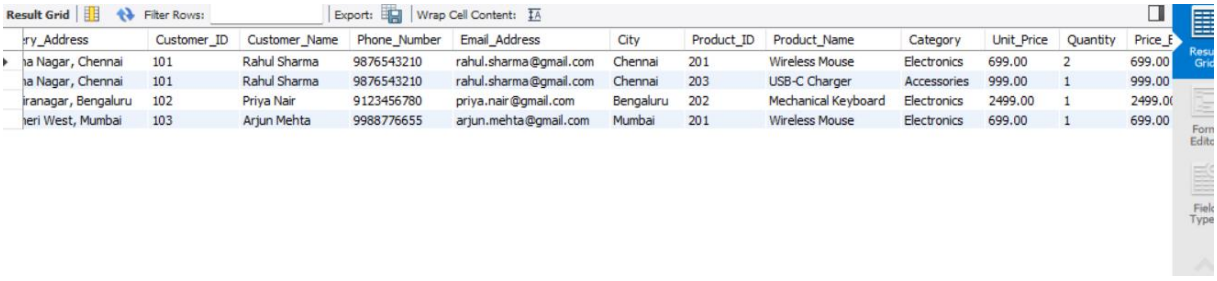
Order_ID	Order_Date	Order_Status	Delivery_Address	Customer_ID	Customer_Name	Phone_Number	Email_Address	City	Product_ID	Product_Name
301	2026-07-20	Delivered	12 Anna Nagar, Chennai	101	Rahul Sharma	9876543210	rahul.sharma@gmail.com	Chennai	201	Wireless Mouse
301	2026-07-20	Delivered	12 Anna Nagar, Chennai	101	Rahul Sharma	9876543210	rahul.sharma@gmail.com	Chennai	203	USB-C Charger
302	2026-07-22	Shipped	45 Indiranagar, Bengaluru	102	Priya Nair	9123 9123456780	riya.nair@gmail.com	Bengaluru	202	Mechanical Keyboard
303	2026-07-24	Processing	8 Andheri West, Mumbai	103	Arjun Mehta	9988776655	arjun.mehta@gmail.com	Mumbai	201	Wireless Mouse

ENORMALIZ... ×

Output

Action Output

#	Time	Action	Message
8	20:56:47	SELECT * FROM FRAUD_REPORT_DENORM ORDER BY Transaction_ID LIMIT 0, 1000	4 row(s) returned
9	21:19:09	USE ProductOrderDB	0 row(s) affected
10	21:19:09	CREATE TABLE ORDER_DETAILS_DENORMALIZED (Order_ID INT, Order_Date DATE, ...	0 row(s) affected
11	21:19:09	INSERT INTO ORDER_DETAILS_DENORMALIZED VALUES (301, '2026-07-20', 'Delivered', '12 Anna Naga...	4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0
12	21:19:09	SELECT * FROM ORDER_DETAILS_DENORMALIZED LIMIT 0, 1000	4 row(s) returned



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ry_Address	Customer_ID	Customer_Name	Phone_Number	Email_Address	City	Product_ID	Product_Name	Category	Unit_Price	Quantity	Price_E
a Nagar, Chennai	101	Rahul Sharma	9876543210	rahul.sharma@gmail.com	Chennai	201	Wireless Mouse	Electronics	699.00	2	699.00
a Nagar, Chennai	101	Rahul Sharma	9876543210	rahul.sharma@gmail.com	Chennai	203	USB-C Charger	Accessories	999.00	1	999.00
iranagar, Bengaluru	102	Priya Nair	9123456780	priya.nair@gmail.com	Bengaluru	202	Mechanical Keyboard	Electronics	2499.00	1	2499.00
eri West, Mumbai	103	Arjun Mehta	9988776655	arjun.mehta@gmail.com	Mumbai	201	Wireless Mouse	Electronics	699.00	1	699.00

NORMALIZATION FUNCTIONS :-

1NF, 2NF, 3NF:-

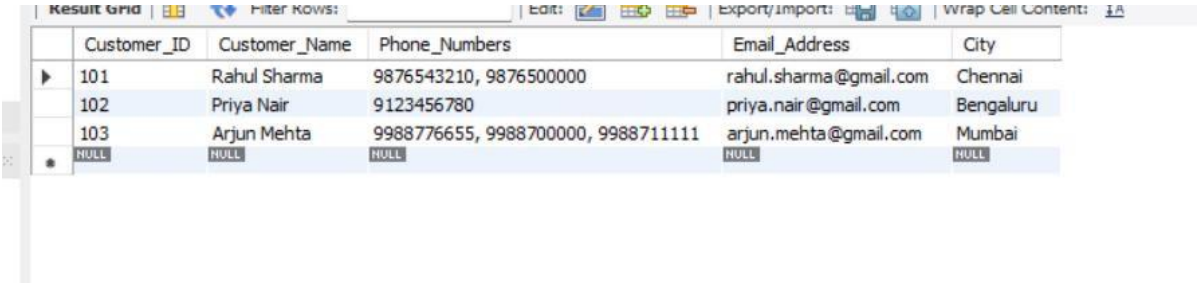
1NF ERROR CODE:-

INTRODUCING ERROR :-

```
-- ================================================================
-- 1NF VIOLATION AND CORRECTION
-- Table: CUSTOMERS
-- ================================================================
USE ProductOrderDB;
-- ----------------------------------------------------------------
-- STEP 1: INTRODUCE THE 1NF VIOLATION
-- Violation type: Multiple values stored in a single column
-- (non-atomic attribute / repeating group)
-- ----------------------------------------------------------------
CREATE TABLE CUSTOMERS_BAD (
```

```
INSERT INTO CUSTOMERS_BAD VALUES
(101, 'Rahul Sharma', '9876543210, 9876500000',           'rahul.sharma@gmail.com', 'Chennai'),
(102, 'Priya Nair',   '9123456780',                        'priya.nair@gmail.com',   'Bengaluru'),
(103, 'Arjun Mehta',  '9988776655, 9988700000, 9988711111', 'arjun.mehta@gmail.com',  'Mumbai');
SELECT * FROM CUSTOMERS_BAD;
-- Problem: Phone_Numbers holds more than one value per row.
-- You cannot search, filter, or index a single phone number cleanly.
-- ----------------------------------------------------------------
-- STEP 2: CORRECT THE VIOLATION
-- Fix: split the repeating group into a separate child table
```

RESULT :- ERROR INTRODUCED



Customer_ID	Customer_Name	Phone_Numbers	Email_Address	City
101	Rahul Sharma	9876543210, 9876500000	rahul.sharma@gmail.com	Chennai
102	Priya Nair	9123456780	priya.nair@gmail.com	Bengaluru
103	Arjun Mehta	9988776655, 9988700000, 9988711111	arjun.mehta@gmail.com	Mumbai
NULL	NULL	NULL	NULL	NULL

ERROR SOLUTION : -

```
-- ------------------------------------------------------------
-- STEP 2: CORRECT THE VIOLATION
-- Fix: split the repeating group into a separate child table
-- so every column holds exactly ONE atomic value
-- ------------------------------------------------------------
CREATE TABLE CUSTOMERS_FIXED (
    Customer_ID     INT PRIMARY KEY,
    Customer_Name   VARCHAR(50),
    Email_Address   VARCHAR(60),
    City            VARCHAR(30)
);
```

```
INSERT INTO CUSTOMERS_FIXED VALUES
(101, 'Rahul Sharma', 'rahul.sharma@gmail.com', 'Chennai'),
(102, 'Priya Nair',   'priya.nair@gmail.com',   'Bengaluru'),
(103, 'Arjun Mehta',  'arjun.mehta@gmail.com',  'Mumbai');
```

```
CREATE TABLE CUSTOMER_PHONES (
    Phone_ID        INT PRIMARY KEY AUTO_INCREMENT,
    Customer_ID     INT,
    Phone_Number    VARCHAR(15),
    FOREIGN KEY (Customer_ID) REFERENCES CUSTOMERS_FIXED(Customer_ID),
    UNIQUE (Customer_ID, Phone_Number)   -- prevents duplicate rows on re-insert
);
```

```
INSERT INTO CUSTOMER_PHONES (Customer_ID, Phone_Number) VALUES
(101, '9876543210'),
(101, '9876500000'),
(102, '9123456780'),
(103, '9988776655'),
(103, '9988700000'),
(103, '9988711111');
```

NEW FIXED RESULT :-

```
-- STEP 3: VERIFY THE FIX
-- ------------------------------------------------------------
SELECT * FROM CUSTOMERS_FIXED;
SELECT * FROM CUSTOMER_PHONES;
-- Confirm 1NF: each customer's phone numbers now
-- appear as separate atomic rows, joined via Customer_ID
SELECT C.Customer_Name, P.Phone_Number
FROM CUSTOMERS_FIXED C
JOIN CUSTOMER_PHONES P
    ON C.Customer_ID = P.Customer_ID
ORDER BY C.Customer_Name;
```

OUTPUT :-

Result Grid | Filter Rows: | Edit: | Export/Import:

Customer_ID	Customer_Name	Email_Address	City
101	Rahul Sharma	rahul.sharma@gmail.com	Chennai
102	Priya Nair	priya.nair@gmail.com	Bengaluru
103	Arjun Mehta	arjun.mehta@gmail.com	Mumbai
NULL	NULL	NULL	NULL

Phone_ID	Customer_ID	Phone_Number
1	101	9876543210
2	101	9876500000
3	102	9123456780
4	103	9988776655
5	103	9988700000
6	103	9988711111

Arjun Mehta	9988776655
Arjun Mehta	9988700000
Arjun Mehta	9988711111

Priya Nair	9123456780

Rahul Sharma	9876543210
Rahul Sharma	9876500000

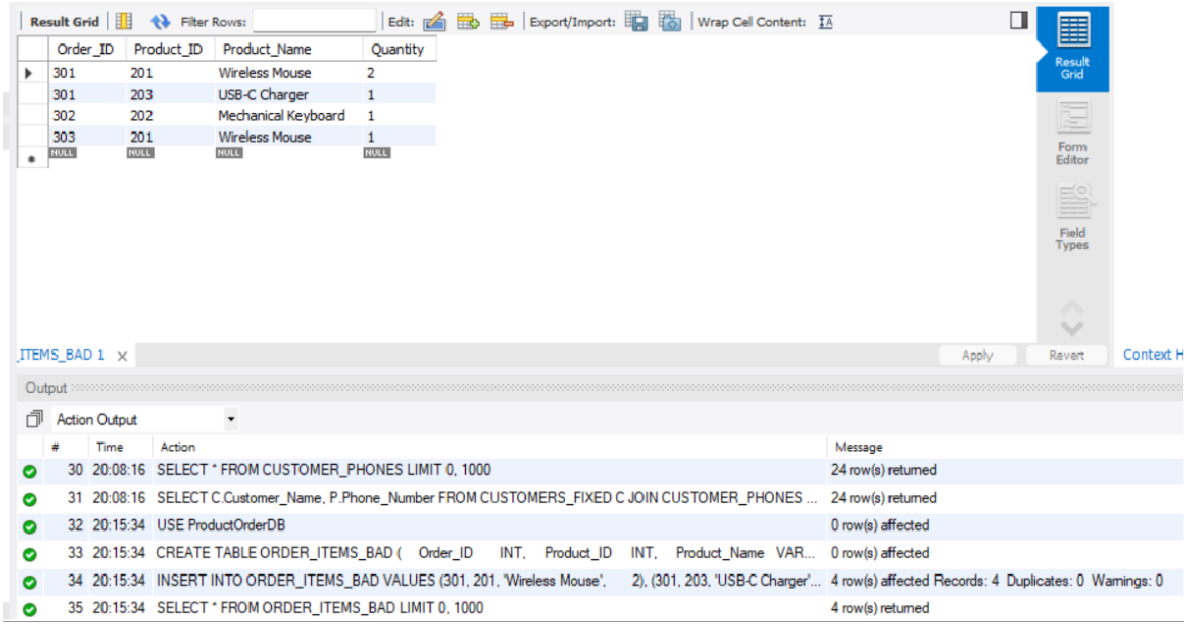
2NF ERROR CODE:-

```
-- ============================================================
-- 2NF VIOLATION AND CORRECTION
-- Table: ORDER_ITEMS
-- ============================================================
USE ProductOrderDB;
-- ------------------------------------------------------------
-- STEP 1: INTRODUCE THE 2NF VIOLATION
-- Violation type: Partial dependency
-- (a non-key attribute depends on only PART of a composite key)
-- ------------------------------------------------------------
CREATE TABLE ORDER_ITEMS_BAD (
```

```sql
CREATE TABLE ORDER_ITEMS_BAD (
    Order_ID       INT,
    Product_ID     INT,
    Product_Name   VARCHAR(50),   -- depends ONLY on Product_ID, not on the full key
    Quantity       INT,
    PRIMARY KEY (Order_ID, Product_ID)   -- COMPOSITE key
);
INSERT INTO ORDER_ITEMS_BAD VALUES
(301, 201, 'Wireless Mouse',       2),
(301, 203, 'USB-C Charger',        1),
```

```sql
(301, 201, 'Wireless Mouse',       2),
(301, 203, 'USB-C Charger',        1),
(302, 202, 'Mechanical Keyboard',  1),
(303, 201, 'Wireless Mouse',       1);
SELECT * FROM ORDER_ITEMS_BAD;
-- Problem: The composite key is (Order_ID, Product_ID).
-- Product_Name depends ONLY on Product_ID — not on Order_ID, and not
-- on the combination of both. This is a PARTIAL DEPENDENCY.
-- Proof: 'Wireless Mouse' is repeated for Product_ID 201 in both
-- order 301 and order 303 — redundant data that can go out of sync
-- if the product name is ever updated in one row but not the other.
```

RESULT : -



ERROR CORRECTION :-

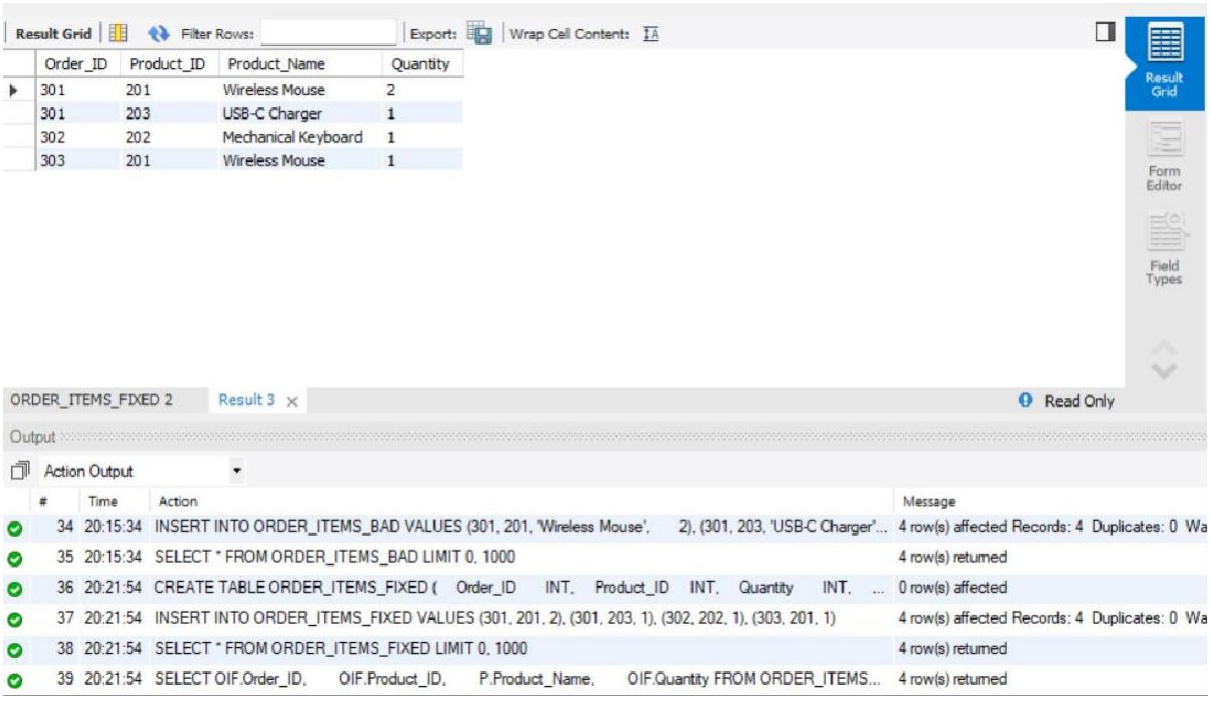
```sql
-- ------------------------------------------------------------
-- STEP 2: CORRECT THE VIOLATION
-- Fix: move the partially-dependent attribute (Product_Name)
-- out into the table it truly depends on (PRODUCTS),
-- leaving only Order_ID/Product_ID/Quantity here
-- ------------------------------------------------------------
CREATE TABLE ORDER_ITEMS_FIXED (
    Order_ID        INT,
    Product_ID      INT,
    Quantity        INT,
    Product_ID      INT,
    Quantity        INT,
    PRIMARY KEY (Order_ID, Product_ID),
    FOREIGN KEY (Product_ID) REFERENCES PRODUCTS(Product_ID)
);
INSERT INTO ORDER_ITEMS_FIXED VALUES
(301, 201, 2),
(301, 203, 1),
(302, 202, 1),
(303, 201, 1);
-- Product_Name now lives ONLY in PRODUCTS (single source of truth),
```

```sql
(303, 201, 1);
-- Product_Name now lives ONLY in PRODUCTS (single source of truth),
-- fetched via JOIN whenever needed — no duplication, no risk of
-- mismatched product names across rows.
-- ------------------------------------------------------------
-- STEP 3: VERIFY THE FIX
-- ------------------------------------------------------------
SELECT * FROM ORDER_ITEMS_FIXED;
-- Confirm 2NF: get product names via JOIN instead of storing them redundantly
SELECT OIF.Order_ID,
       OIF.Product_ID,
```

NEW FIXED SOLUTION:-

```sql
SELECT * FROM ORDER_ITEMS_FIXED;
-- Confirm 2NF: get product names via JOIN instead of storing them redundantly
SELECT OIF.Order_ID,
       OIF.Product_ID,
       P.Product_Name,
       OIF.Quantity
FROM ORDER_ITEMS_FIXED OIF
JOIN PRODUCTS P
    ON OIF.Product_ID = P.Product_ID
ORDER BY OIF.Order_ID;
```

OUTPUT:-



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Order_ID	Product_ID	Product_Name	Quantity
301	201	Wireless Mouse	2
301	203	USB-C Charger	1
302	202	Mechanical Keyboard	1
303	201	Wireless Mouse	1

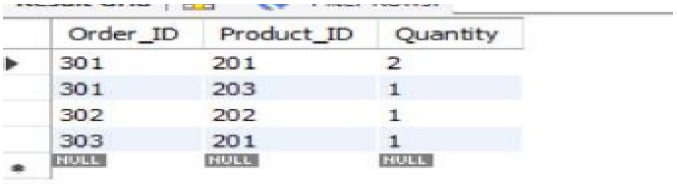
Result Grid | Form Editor | Field Types

ORDER_ITEMS_FIXED 2 | Result 3 | Read Only

Output

Action Output

#	Time	Action	Message
34	20:15:34	INSERT INTO ORDER_ITEMS_BAD VALUES (301, 201, 'Wireless Mouse', 2), (301, 203, 'USB-C Charger'...	4 row(s) affected Records: 4 Duplicates: 0 Wa
35	20:15:34	SELECT * FROM ORDER_ITEMS_BAD LIMIT 0, 1000	4 row(s) returned
36	20:21:54	CREATE TABLE ORDER_ITEMS_FIXED (Order_ID INT, Product_ID INT, Quantity INT, ...	0 row(s) affected
37	20:21:54	INSERT INTO ORDER_ITEMS_FIXED VALUES (301, 201, 2), (301, 203, 1), (302, 202, 1), (303, 201, 1)	4 row(s) affected Records: 4 Duplicates: 0 Wa
38	20:21:54	SELECT * FROM ORDER_ITEMS_FIXED LIMIT 0, 1000	4 row(s) returned
39	20:21:54	SELECT OIF.Order_ID, OIF.Product_ID, P.Product_Name, OIF.Quantity FROM ORDER_ITEMS...	4 row(s) returned



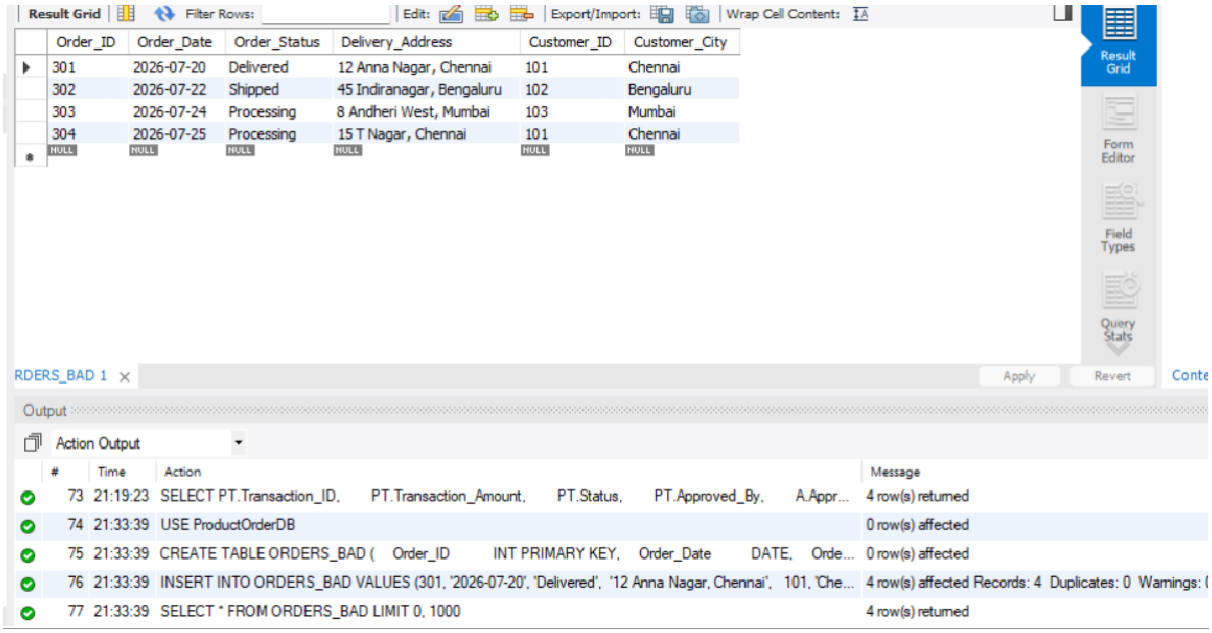
Order_ID	Product_ID	Quantity
301	201	2
301	203	1
302	202	1
303	201	1
NULL	NULL	NULL

ORDER_ITEMS_FIXED 2 | Result 3

3NF ERROR CODE:-

```
-- 3NF VIOLATION AND CORRECTION
-- Database: ProductOrderDB
-- Table: ORDERS
USE ProductOrderDB;
-- STEP 1: INTRODUCE THE 3NF VIOLATION
-- Violation type: Transitive dependency
-- (a non-key attribute depends on ANOTHER non-key attribute,
--  not directly on the primary key)
CREATE TABLE ORDERS_BAD (
    Order_ID           INT PRIMARY KEY,
    Order_Date          DATE,
    Order_Status        VARCHAR(20),
    Delivery_Address    VARCHAR(100),
    Customer_ID         INT,
    Customer_City       VARCHAR(30),   -- depends on Customer_ID, NOT on Order_ID
    FOREIGN KEY (Customer_ID) REFERENCES CUSTOMERS(Customer_ID)
);
INSERT INTO ORDERS_BAD VALUES
(301, '2026-07-20', 'Delivered',  '12 Anna Nagar, Chennai',   101, 'Chennai'),
(302, '2026-07-22', 'Shipped',    '45 Indiranagar, Bengaluru', 102, 'Bengaluru'),
(303, '2026-07-24', 'Processing', '8 Andheri West, Mumbai',    103, 'Mumbai'),
(304, '2026-07-25', 'Processing', '15 T Nagar, Chennai',       101, 'Chennai');
SELECT * FROM ORDERS_BAD;
-- Problem: The primary key is Order_ID.
-- Customer_City does NOT depend directly on Order_ID —
-- it depends on Customer_ID (which customer placed the order).
-- This chain is: Order_ID -> Customer_ID -> Customer_City
-- That's a TRANSITIVE DEPENDENCY, which violates 3NF.
-- Proof: Customer_ID 101 -> 'Chennai' is repeated in orders 301 and 304.
-- If Rahul Sharma (customer 101) moves cities, you'd have to update
-- EVERY order row he ever placed — a classic update anomaly.
-- (Note: this is different from Delivery_Address, which legitimately
-- varies per order and correctly belongs at the Order_ID level.)
```

RESULT:-



Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

Order_ID	Order_Date	Order_Status	Delivery_Address	Customer_ID	Customer_City
301	2026-07-20	Delivered	12 Anna Nagar, Chennai	101	Chennai
302	2026-07-22	Shipped	45 Indiranagar, Bengaluru	102	Bengaluru
303	2026-07-24	Processing	8 Andheri West, Mumbai	103	Mumbai
304	2026-07-25	Processing	15 T Nagar, Chennai	101	Chennai
NULL	NULL	NULL	NULL	NULL	NULL

Result Grid | Form Editor | Field Types | Query Stats

RDERS_BAD 1 | Apply | Revert | Conte

Output

Action Output

#	Time	Action	Message
73	21:19:23	SELECT PT.Transaction_ID, PT.Transaction_Amount, PT.Status, PT.Approved_By, A.Appr...	4 row(s) returned
74	21:33:39	USE ProductOrderDB	0 row(s) affected
75	21:33:39	CREATE TABLE ORDERS_BAD (Order_ID INT PRIMARY KEY, Order_Date DATE, Orde...	0 row(s) affected
76	21:33:39	INSERT INTO ORDERS_BAD VALUES (301, '2026-07-20', 'Delivered', '12 Anna Nagar, Chennai', 101, 'Che...	4 row(s) affected Records: 4 Duplicates: 0 Warnings: (
77	21:33:39	SELECT * FROM ORDERS_BAD LIMIT 0, 1000	4 row(s) returned

ERROR SOLUTION:-

```
-- STEP 2: CORRECT THE VIOLATION
-- Fix: remove the transitively-dependent attribute (Customer_City)
-- from ORDERS — it already lives correctly in CUSTOMERS,
-- where Customer_ID is the primary key it truly depends on
-- ----------------------------------------------------------------
CREATE TABLE ORDERS_FIXED (
    Order_ID          INT PRIMARY KEY,
    Order_Date         DATE,
    Order_Status        VARCHAR(20),
    Delivery_Address    VARCHAR(100),
    Customer_ID          INT,
    FOREIGN KEY (Customer_ID) REFERENCES CUSTOMERS(Customer_ID)
);
INSERT INTO ORDERS_FIXED VALUES
(301, '2026-07-20', 'Delivered',  '12 Anna Nagar, Chennai',    101),
(302, '2026-07-22', 'Shipped',    '45 Indiranagar, Bengaluru', 102),
(303, '2026-07-24', 'Processing', '8 Andheri West, Mumbai',    103),
(304, '2026-07-25', 'Processing', '15 T Nagar, Chennai',       101);
-- Customer_City now lives ONLY in CUSTOMERS (single source of truth).
-- Updating Rahul Sharma's city now takes ONE update, not two.
```

NEW FIXED SOLUTION:-

```
6
7   -- STEP 3: VERIFY THE FIX
8   SELECT * FROM ORDERS_FIXED;
9   -- Confirm 3NF: get customer city via JOIN instead of storing it redundantly
0   SELECT O.Order_ID,
1          O.Order_Date,
2          O.Order_Status,
3          O.Delivery_Address,
4          C.Customer_Name,
5          C.City AS Customer_City
6   FROM ORDERS_FIXED O
7   JOIN CUSTOMERS C
8       ON O.Customer_ID = C.Customer_ID
9   ORDER BY O.Order_ID;
```

OUTPUT:-

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

Order_ID	Order_Date	Order_Status	Delivery_Address	Customer_ID
301	2026-07-20	Delivered	12 Anna Nagar, Chennai	101
302	2026-07-22	Shipped	45 Indiranagar, Bengaluru	102
303	2026-07-24	Processing	8 Andheri West, Mumbai	103
304	2026-07-25	Processing	15 T Nagar, Chennai	101
NULL	NULL	NULL	NULL	NULL

ORDERS_FIXED 2 | Result 3

Output

Action Output

#	Time	Action	Message
77	21:33:39	SELECT * FROM ORDERS_BAD LIMIT 0, 1000	4 row(s) returned
78	21:40:02	CREATE TABLE ORDERS_FIXED (Order_ID INT PRIMARY KEY, Order_Date DATE, Or...	0 row(s) affected
79	21:40:02	INSERT INTO ORDERS_FIXED VALUES (301, '2026-07-20', 'Delivered', '12 Anna Nagar, Chennai', 101), (...	4 row(s) affected Record
80	21:40:02	SELECT * FROM ORDERS_FIXED LIMIT 0, 1000	4 row(s) returned
81	21:40:02	SELECT O.Order_ID, O.Order_Date, O.Order_Status, O.Delivery_Address, C.Customer_Nam...	4 row(s) returned

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Order_ID	Order_Date	Order_Status	Delivery_Address	Customer_Name	Customer_City
301	2026-07-20	Delivered	12 Anna Nagar, Chennai	Rahul Sharma	Chennai
302	2026-07-22	Shipped	45 Indiranagar, Bengaluru	Priya Nair	Bengaluru
303	2026-07-24	Processing	8 Andheri West, Mumbai	Arjun Mehta	Mumbai
304	2026-07-25	Processing	15 T Nagar, Chennai	Rahul Sharma	Chennai

---